

EV Range Prediction

Overview — Problem & Objective

The goal is to predict an electric vehicle's official driving range (range_km) from its static specification sheet — battery, performance, charging, dimensions, drivetrain, body type, seating, cargo and towing fields — without using any trip-level telemetry, since none is available in this dataset.

Target variable: range_km (official driving range, km).

Key restriction respected throughout: efficiency_wh_per_km was never used as a model input. Efficiency and range are algebraically linked through battery capacity, so including it would let the model reconstruct the target instead of genuinely learning from specifications. It was used only for EDA / sanity-checking.

Dataset: 478 EV models from 59 brands, one row per model, 22 columns covering battery, performance, charging, dimensions, drivetrain, body type, seating, cargo and towing specifications.

1. Data Cleaning & Preprocessing

REQUIREMENT: Data cleaning and preprocessing

1.1 Data Quality Issues Identified During Profiling

- battery_type is constant (Lithium-ion for all 478 rows) — zero variance, no predictive value.
- fast_charge_port is almost entirely CCS (476 of 478 rows) — near-zero variance.
- source_url is listing metadata, not a specification.
- number_of_cells is missing in 202 of 478 rows (~42.3%) — too unreliable to impute confidently as a trusted numeric value on its own.
- torque_nm missing in 7 rows (~1.5%); fast_charging_power_kw_dc missing in 1 row; towing_capacity_kg missing in 26 rows (~5.4%) — all low enough to safely impute.
- cargo_volume_l contains 3 non-numeric entries recorded as "N Banana Boxes" instead of litres (Audi Q6 e-tron quattro, Maxus MIFA 9, Mercedes-Benz EQS SUV 580 4MATIC).
- model has 1 missing value.
- No exact duplicate rows were found.

1.2 Cleaning Decisions & Justification

Decision	Why
Convert "N Banana Boxes" entries to litres using 1 box ≈ 52 L	Documented approximation; preserves 3 real data points instead of discarding them
Fill missing model with "Unknown"	Identifier only, not used as a model feature

Decision	Why
Create <code>cells_missing_flag</code> before imputing <code>number_of_cells</code>	Lets the model learn whether "unknown cell count" itself is informative (e.g. brand under-reporting) instead of silently masking it
Drop <code>battery_type</code> , <code>fast_charge_port</code> , <code>source_url</code>	Zero / near-zero variance or non-specification metadata — no predictive signal
Defer imputation of remaining numeric gaps (<code>number_of_cells</code> , <code>torque_nm</code> , <code>fast_charging_power_kw_dc</code> , <code>towing_capacity_kg</code>) into the modelling pipeline	Imputing before the train/test split would leak test-set distribution into training; fitting the imputer on the training fold only avoids this

1.3 Preprocessing Pipeline

- Median imputation + scaling for numeric features.
- Most-frequent imputation + one-hot encoding for categorical features.
- Both wrapped in a single scikit-learn `ColumnTransformer` inside the Pipeline, fit only on the training fold, so the exact same transformations replay on any new input at prediction time.

2. Exploratory Data Analysis

REQUIREMENT: Exploratory data analysis

- `range_km` is roughly unimodal and right-skewed, with a long tail toward large-battery / high-performance vehicles. No outliers were removed since these are genuine vehicles the model must generalise to.
- `battery_capacity_kWh` is the strongest single numeric correlate of range, as expected physically.
- `efficiency_wh_per_km` also correlates strongly with range — this confirms, rather than contradicts, the decision to exclude it as a feature (it is a near-restatement of the target via battery capacity).
- AWD vehicles trend toward higher battery capacity and range on average, but with wide spread — drivetrain alone is a weak predictor and needs to combine with size/battery features.
- SUVs and larger body styles show higher median range than compact hatchbacks, consistent with larger battery packs — `car_body_type` is a useful categorical feature.

3. Feature Engineering

REQUIREMENT: Feature engineering

Beyond raw columns, the following physically-motivated features were engineered:

Feature	Rationale
<code>footprint_m2</code> (length × width)	Proxy for overall vehicle size / aerodynamic drag area
<code>volume_m3</code> (footprint × height)	Coarse proxy for interior / cabin volume

Feature	Rationale
battery_per_seat	Battery capacity normalised by seating — captures "battery generosity" relative to vehicle purpose
torque_per_100kwh	Torque normalised by battery size — proxy for performance-tuned vs. range-tuned drivetrains
segment_group	Letter-prefix rollup of the 15 fine-grained market segments into broader classes
cells_missing_flag	Whether number_of_cells was reported at all (see Section 1)

number_of_cells itself was retained as an optional numeric feature (its high missingness caps its usefulness) rather than dropped outright, per the challenge's guidance.

4. Model Development

REQUIREMENT: Model development

4.1 Train / Test Split & Pipeline

- brand and model excluded (identifiers, not specifications a judge would enter into the tester).
- efficiency_wh_per_km excluded (target leakage, see Overview).
- 80/20 train/test split, random_state = 42 for full reproducibility.

4.2 Model Comparison

Six candidate models were compared with 5-fold cross-validation on the training set only: Linear Regression, Ridge, Lasso, Random Forest, Gradient Boosting and XGBoost. A deep neural network was intentionally excluded — with only 478 rows, it offers no expected benefit over these classical methods and is prone to overfitting.

4.3 Hyperparameter Tuning (GridSearchCV, 5-fold CV)

Model	Best parameters found
Ridge	alpha = 1 (CV RMSE $\approx$ 21.92 km)
Gradient Boosting	n_estimators = 300, max_depth = 3, learning_rate = 0.1 (CV RMSE $\approx$ 21.37 km)

4.4 Final Model Selection

Final model selected: Gradient Boosting Regressor (tuned). It achieves lower test MAE/RMSE and higher R^2 than Ridge, and can capture non-linear interactions between battery capacity, size and drivetrain that a linear model cannot, without overfitting given its shallow, regularised trees on this dataset size. Ridge is retained in the notebook as a simpler, more interpretable fallback.

5. Model Evaluation

REQUIREMENT: Model evaluation

5.1 Final Held-Out Test Set Results

The test set was untouched during model selection and tuning — these are genuine out-of-sample numbers:

Model	MAE (km)	RMSE (km)	R ²
Ridge (tuned)	16.72	21.11	0.9579
Gradient Boosting (tuned)	10.97	15.09	0.9785

5.2 Feature Importance

The top contributors to the final Gradient Boosting model's predictions, in order:

- battery_per_seat — dominant driver ($\approx 50\%$ of total importance)
- battery_capacity_kWh — second largest contributor ($\approx 34\%$)
- height_mm, fast_charging_power_kw_dc, torque_per_100kwh, and car_body_type (SUV / Sedan) contribute smaller, physically sensible amounts.

This confirms the model is relying on genuine specification features consistent with domain intuition, and not on anything resembling the excluded efficiency column.

5.3 Evaluation Sanity Checks

- Predicted-vs-actual scatter plot tracks the ideal diagonal closely across the full range of values.
- Residuals scatter evenly around zero with no funnel shape — error does not grow systematically with vehicle range, i.e. the model isn't only accurate for one market segment.

6. Reproducibility

REQUIREMENT: Reproducibility

- Random seed fixed at 42 everywhere a stochastic step occurs (split, cross-validation folds, model initialisation).
- The complete pipeline (preprocessing + tuned model) is saved as `ev_range_pipeline.joblib` — reload-verified to reproduce identical predictions.
- All preprocessing steps are fit only on the training fold and wrapped into the same Pipeline object as the model, so the identical transformation sequence replays on any new input at prediction time.

7. Technical Documentation

REQUIREMENT: Technical documentation

This report itself constitutes the technical documentation of the solution, covering the problem framing, dataset, cleaning decisions, EDA findings, feature engineering, modelling pipeline, evaluation results and known limitations end-to-end, so the approach can be understood and audited without reading the source code.

7.1 Limitations

- This is a specification-based model, not a real-time range estimator: it has no access to trip-level telemetry (traffic, weather, HVAC load, State of Charge / State of Health), so it predicts the manufacturer-rated range only.
- `number_of_cells` is missing for ~42% of rows; its contribution to the final model is small, limiting the practical impact of this gap.
- With 478 training rows, performance on future EV designs well outside the specification ranges seen here (e.g. much larger batteries) is less certain.
- The 1 banana box $\approx$ 52 L conversion is a documented approximation affecting only 3 rows, not an official unit conversion.

8. Working Interactive Demonstration

REQUIREMENT: Working interactive demonstration

An interactive website broadcasts predictions after specification entry, without touching the underlying code, using the saved pipeline described in Sections 4 and 6.

[Live demo -> bayesian-blueprints-range-predictor.vercel.app](https://bayesian-blueprints-range-predictor.vercel.app)